ct-02 — AtCoder Beginner Contest 476

Link	https://atcoder.jp/contests/abc476/tasks
Date attempted	not recorded — solution files appeared 2026-09-22
Problems	7 (A–G)
Solved in contest	not recorded
Solved after	not recorded — A, B, C have working solutions; timing unknown
Unsolved	D, E, F, G — not attempted (link-only stubs)
Concepts	rolling order statistics, exchange argument, monotone prefix budget, segment tree argmax, chebyshev-manhattan rotation, weighted absolute-difference sums, dilworth's theorem, aligned block decomposition, stream flushing cost

1. The set

#	problem	verdict	key concept	one-line idea
A	Appender	AC	string building	append `r` if the last character is `e`, else `er`
B	Wild Card	AC	position-wise matching	`T` matches `S` iff every non-`*` position agrees; `*` is a free pass
C	Third Largest Number	AC	rolling order statistics	the 3rd largest of every prefix — three scalars suffice, no heap
D	Automat	not attempted	exchange argument	change makes money fungible; only the K-bill count is a second constraint. Sort both lists, sweep the drink count
E	Min-Max Swap	not attempted	segment tree argmax	two segment trees over `(value, index)`; each op is 2 queries + 4 point updates
F	Chebyshev Cafe	not attempted	chebyshev ↔ manhattan rotation	`max(|dr|,|dc|) = (|du|+|dv|)/2` with `u=r+c, v=r−c`, which separates a 2D kernel into two 1D sums
G	Increasing Popcount	not attempted	dilworth's theorem	answer = longest non-increasing-popcount subsequence on `[L,R]`; count it over `O(log R)` aligned blocks

Solution files: `a.cpp`, `b.cpp`, `c.cpp` carry working solutions. `d.cpp`, `e.cpp`, `f.cpp`, `g.cpp` contain only the problem link — problems not attempted, recorded honestly rather than omitted.

Every algorithm written down in §3 was run against the official samples before being documented; D was additionally cross-checked against an exhaustive subset-enumeration brute force on 300 random small inputs, and G's reformulation was checked against an independent minimum-chain-cover computation and against sample 4 (`L=100, R=10^7 → 1636985`).

---

2. Key concepts across the set

2.1 Rolling order statistics under append — used in C

What it is: maintaining the `k`-th largest element of a sequence that only ever grows at the right end, reporting it after every append. The `k` here is a small compile-time constant (3), which is what makes the cheap solution possible.

How it works: the general tool is a heap or a balanced BST, and both are `O(log n)` per element. But when `k` is a fixed small constant, the entire state you need is the top `k` values seen so far, held in `k` scalars. On append, the new element either displaces one of them or is discarded, and the update is a `k`-step cascade:

```
if      (x > m1) { m3 = m2; m2 = m1; m1 = x; }
else if (x > m2) { m3 = m2; m2 = x; }
else if (x > m3) { m3 = x; }
```

The strict `>` chain is what makes duplicates behave: equal values fall through to the first slot they genuinely beat, so `[5,5,5]` correctly ends as `m1=m2=m3=5`. This is a *multiset* third-largest, not a third-*distinct* value, which matches the statement's "sorted in descending order, take the third".

Complexity: `O(1)` time and `O(1)` extra space per element, versus `O(log n)` time and `O(n)` space for the heap.

How it showed up here: `c.cpp` uses the heap form — `push`, then pop twice, read `top()`, push the two back. That is `O(n log n)` and correct, but it keeps all `n` elements alive and allocates a `std::vector` per iteration. The scalar form removes both.

The recognisable signal: "for every prefix, report the `k`-th largest" with `k` a literal in the statement. A `k` that is itself an input would flip this back to a heap (keep a size-`k` min-heap) — that is the version worth knowing, because the constant-`k` trick does not generalise.

Sources:

- cppreference — std::priority_queue — "constant time lookup of the largest element, at the expense of logarithmic insertion and extraction"

2.2 One sort order that minimises two costs at once — used in D

What it is: a selection problem with two independent budgets. Normally choosing which `r` items to take is a trade-off — cheapest in one currency is not cheapest in the other — and the problem becomes a knapsack. It collapses when one cost is a **monotone non-decreasing function of the other**.

How it works: if `g` is non-decreasing and you must pick `r` items to minimise both `Σ price` and `Σ g(price)`, then the `r` cheapest items by price minimise both simultaneously. There is no trade-off to resolve: any exchange that lowers one sum also lowers (or holds) the other. This is the exchange argument in its simplest form — swap any chosen item for a cheaper unchosen one and neither sum can increase.

Complexity: `O(m log m)` for the sort, after which every prefix is optimal for its own length.

How it showed up here: a drink of price `B_j` costs `ceil(B_j / K)` K-dollar bills. `ceil(·/K)` is non-decreasing in `B_j`, so sorting the drinks ascending by price makes prefix `r` minimise both

the money spent *and* the K-bills consumed. That is the whole reason a single sweep over `r` is valid instead of a 2D DP.

The recognisable signal: two resource constraints where one resource's cost is a rounded-up or bucketed version of the other. Check monotonicity first — if it holds, the problem is a sweep; if it does not, you are in knapsack territory.

Sources:

- USACO Guide — Greedy Algorithms with Sorting — the exchange-argument style of correctness proof

2.3 Monotone prefix budgets: binary search, then two pointers — used in D

What it is: once both lists are sorted and prefix-summed, "how many of the cheapest desserts fit in budget `W`" is a search for the last prefix sum `≤ W` on a non-decreasing array.

How it works: prefix sums of positive values are strictly increasing, which is exactly the partition the STL's binary searches require — `upper_bound(preA.begin(), preA.end(), W) - preA.begin() - 1` is the count. As the drink count `r` rises, the leftover budget falls monotonically, so the dessert count falls monotonically too: the binary search can be replaced by a pointer that only ever moves left, giving `O(n)` for the sweep and leaving the sort as the dominant cost.

Complexity: `O((N+M) log(N+M))` either way — the binary search is not the bottleneck, so reach for it first and only switch to two pointers if the sort is also gone.

How it showed up here: for each feasible drink count `r` (those with `preK[r] ≤ Y`), the dessert count is a search on `preA` against `X + K·Y − preB[r]`.

The recognisable signal: an answer of the form `max over r of (r + f(r))` where `f` is monotone in `r`. That is a two-pointer sweep wearing a disguise.

Sources:

- USACO Guide — Binary Search — "binary search on the answer only works if the answer function is monotonic"

2.4 A segment tree that returns an argmax, not a max — used in E

What it is: the standard range-max segment tree returns the *value* of the maximum. When you need to modify the element that attained it, you need its *position* as well.

How it works: store a pair `(value, index)` in every node and make the merge operator pick the pair with the larger value — the index rides along for free. The identity element is a pair that always loses: `(-1, -1)` for a max tree over positive values, `(INT_MAX, -1)` for a min tree. Nothing about the tree's structure changes; only the monoid does. CP-Algorithms calls this out explicitly as the general pattern — a node may store whatever extra information the merge can maintain.

Complexity: `O(n)` to build, `O(log n)` per query and per point update.

How it showed up here: each operation needs the argmin and argmax of `P[L..R]`, then swaps those two entries. Because the swap changes two positions and both trees index the same array, one operation costs 2 queries and 4 point updates — still `O(log n)`. `P` is a permutation, so both extrema are unique and there is no tie-breaking question.

The recognisable signal: "find the extremum in a range **and then modify it**". The modification is what rules out a static sparse table and forces a segment tree.

Sources:

- CP-Algorithms — Segment Tree — including the "finding the maximum and the number of times it appears" pattern, of which `(value, index)` is a variant

2.5 Chebyshev ↔ Manhattan rotation — used in F

What it is: in two dimensions, the `L∞` (Chebyshev) metric and the `L1` (Manhattan) metric are the same metric seen in rotated, scaled coordinates. The identity is:

```
max(|dr|, |dc|)  =  ( |dr + dc| + |dr − dc| ) / 2
```

Substituting `u = r + c` and `v = r − c` turns a Chebyshev distance in `(r,c)` into half a Manhattan distance in `(u,v)`.

How it works: `max(a,b) = (a + b + |a − b|)/2`; applying it to `a = |dr|`, `b = |dc|` and simplifying the absolute values gives the identity above. The payoff is **separability**: Manhattan distance is a sum of independent per-coordinate terms, while Chebyshev's `max` couples the coordinates and blocks every decomposition you would want to apply.

Complexity: free — it is an algebraic rewrite. What it buys is the reduction of an `O(N²)`-sized 2D problem to two 1D problems of size `O(N)`.

How it showed up here: `f(i,j) = Σ C[r][c] · max(|r−i|, |c−j|)` becomes

```
f(i,j) = ( Σ C[r][c]·|(r+c) − (i+j)| + Σ C[r][c]·|(r−c) − (i−j)| ) / 2
```

Bucket the populations by `u = r+c` into `W1` and by `v = r−c` into `W2`, and each half is a one-dimensional weighted sum of absolute differences (§2.6). Every cell then reads two precomputed 1D arrays, so the whole grid costs `O(N²)`.

A parity trap worth writing down: `du` and `dv` always share a parity, so `|du| + |dv|` is even for each individual term and the halving is exact — but `W1`'s total and `W2`'s total are each frequently odd on their own. Add the two 1D answers first, then divide by 2. Dividing each half separately silently loses a unit per cell.

The recognisable signal: `max(|Δx|, |Δy|)` appearing as a *weight inside a sum* over a grid. Chebyshev as a one-off distance is harmless; Chebyshev summed over every pair is the signal to rotate.

Sources:

- Wikipedia — Chebyshev distance — "the planar Chebyshev distance can be viewed as equivalent by rotation and scaling to the planar Manhattan distance", and the warning that this equivalence does **not** generalise past two dimensions

2.6 Weighted sums of absolute differences in O(1) — used in F

What it is: given weights `w[x]` on integer positions `x`, answer `A(S) = Σ_x w[x]·|x − S|` for many query points `S`.

How it works: split at `S`. Points at or below `S` contribute `S·(Σ w) − (Σ w·x)`; points above contribute `(Σ w·x) − S·(Σ w)`. Two prefix-sum arrays — one over `w`, one over `w·x` — make each query `O(1)` after an `O(1)` boundary lookup, or `O(log n)` if you binary-search the boundary. Sweeping `S` in increasing order makes even the lookup free.

Complexity: `O(n)` preprocessing, `O(1)` per query.

How it showed up here: each of the two rotated axes in §2.5 is exactly this, with `w = W1` (or `W2`) and `S = i+j` (or `i−j`).

The recognisable signal: "total distance from everyone to a candidate point", in one dimension, asked repeatedly. It is also the machinery behind the classic "minimise total distance → the median" result.

Sources:

- USACO Guide — Binary Search (for locating the split point when `S` is not swept in order)

2.7 Dilworth's theorem: minimum chain cover = maximum antichain — used in G

What it is: in any finite partially ordered set, the minimum number of chains needed to cover every element equals the maximum size of an antichain. It is the tool that converts a "how few groups do I need" question into a "how large can one bad set be" question, which is usually far easier to count.

How it works: define a partial order on the indices `L..R` by `i < j` iff `i < j` **and** `popcount(i) < popcount(j)`. The problem says two indices may share a value only when they are comparable in exactly this order, so a *value class* is precisely a **chain**, and the answer — the minimum number of distinct values — is the **minimum chain cover**. Dilworth turns that into the maximum antichain: a set of pairwise-incomparable indices, i.e. a subsequence $x_1 < x_2 < \dots < x_m$ with $\text{popcount}(x_1) \geq \text{popcount}(x_2) \geq \dots \geq \text{popcount}(x_m)$.

So: the answer is the length of the longest non-increasing-popcount subsequence of `[L, R]`.

Complexity: the theorem itself is free; it is a change of question. What it costs is that you must now count a longest non-increasing subsequence over a range of size up to `10^18` — see §2.8.

The trap this replaces. The natural guess is "answer = the size of the largest popcount class in `[L,R]`", since all members of one class are mutually incomparable. It is a valid *lower* bound and it matches samples 1, 2 and 3, which is exactly what makes it dangerous. It is wrong: on `[7, 8]`, popcounts are `(3, 1)`, every class has size 1, but `7` and `8` cannot share a value because `popcount(7) = 3 ≥ 1 = popcount(8)`, so the answer is 2. An antichain may span several classes as long as popcount never rises.

The recognisable signal: "minimum number of labels/colours/groups such that same-labelled items obey a strict ordering constraint" is always a chain cover, and therefore always Dilworth.

Sources:

- Wikipedia — Dilworth's theorem — "the maximum size of an antichain of incomparable elements equals the minimum number of chains needed to cover all elements"
- AtCoder — ABC476 G editorial (English)

2.8 Aligned block decomposition of [L, R] — used in G

What it is: the standard way to make a range up to `10^18` tractable: split `[L, R+1)` into `O(log R)` maximal **aligned** blocks, each of the form `p·2^c + [0, 2^c)`. Every element of such a block shares the same high-bit prefix `p`; the low `c` bits run over all of `[0, 2^c)`.

How it works: within one block, an element's popcount is `popcount(p) + popcount(low bits)`, and the number of elements whose low part has popcount exactly `k` is `C(c, k)`. So a block is fully described by its prefix popcount and its width — the individual numbers never need to be enumerated.

The extra ingredient G needs is a lemma about what one block can contribute: inside a full block `[0, 2^c)`, the longest non-increasing-popcount subsequence has length `max_k C(c, k)` — you cannot beat taking a single popcount class. The editorial proves this with the **Lubell–Yamamoto–Meshalkin inequality**, which bounds `Σ 1/C(c, popcount(xᵢ)) ≤ 1` for an antichain of subsets and is the same machinery behind Sperner's theorem.

With that, the whole query becomes a DP across the `O(log R)` blocks: pick one popcount class per block, with the chosen popcounts non-increasing left to right, maximising the total count.

Complexity: `O(log² R)` per query — roughly 60 blocks × 60 popcount values, about 3600 operations, which is what makes `T ≤ 10^4` comfortable.

The recognisable signal: `R ≤ 10^18` with `T` in the thousands. The per-query budget is polylog, and for anything phrased over binary representations that means aligned blocks plus a DP over bit positions.

Sources:

- Wikipedia — Lubell–Yamamoto–Meshalkin inequality
- AtCoder — ABC476 G editorial (English)

2.9 Output volume: `endl` at 5×10^5 lines — used in C

What it is: `std::endl` writes `'\n'` **and flushes.** At `N = 5×10^5` output lines that is half a million flushes, each a syscall-adjacent operation the stream cannot batch.

How it works: by default the C++ streams are kept in sync with C's `stdio`, which costs on every operation; `std::ios_base::sync_with_stdio(false)` decouples them and lets `cin` / `cout` buffer independently. cppreference notes the synchronised streams are effectively unbuffered, which is the source of the overhead. The two habitual lines are:

```
std::ios_base::sync_with_stdio(false);
std::cin.tie(nullptr);
```

and then `'\n'` instead of `std::endl` everywhere except a genuine interactive flush.

How it showed up here: `c.cpp` emits up to `N−2 ≈ 5×10^5` lines with `std::endl` and no `sync_with_stdio(false)`. It is the one part of that solution with a real chance of costing a verdict rather than style points; the algorithm itself is fine.

The recognisable signal: output lines in the same order of magnitude as `N`, with `N ≥ 10^5`.

Sources:

- cppreference — std::ios_base::sync_with_stdio — "disabling synchronization can provide considerably faster performance"

---

3. From brute force to optimal, problem by problem

A — Appender

One rung. `|S| ≤ 10`; check the last character and append `"r"` or `"er"`.

`a.cpp` is correct. Two cosmetic notes: there is a stray `;` after `main`'s closing brace (legal as an empty declaration, harmless), and `s + "r"` builds a temporary where `s += "r"` would not — irrelevant at this size, a habit worth having at `|S| = 10^6`.

B — Wild Card

- **Rung 1 — enumerate.** Replace each `*` with all 26 letters and compare: `O(26^{#stars} · n)`. Dies at 5 stars.
- **Rung 2 — position-wise, `O(n)`.** Matching is independent per position: a `*` matches anything, any other character must agree. Scan once.

`b.cpp` is rung 2 and is correct — early `break` on mismatch, `continue` on `*`. With `N ≤ 100` nothing here is load-bearing. (AtCoder also published a user editorial doing it with a regex; at this size both are fine, and the manual loop is the one that survives a harder variant where `*` matches *any number* of characters, which is a genuinely different — DP — problem.)

C — Third Largest Number

- **Rung 1 — re-sort every prefix.** For each `k`, sort `A[0..k)` and read index 2: `O(n² log n)`. At `n = 5×10^5` this is ~`10^13` operations. Dead on arrival, but it is the definition, and writing it is how you get a reference to test against.
- **Rung 2 — an ordered container.** Keep a `std::multiset`, insert each element, advance an iterator two steps from `begin()` on the descending view. `O(n log n)`, but iterator advancement on a node-based container is slow and the memory is `O(n)`.
- **Rung 3 — a max-heap (what `c.cpp` does).** Push, pop twice, read `top()`, push the two back. `O(n log n)` with a good constant. **This is correct**, including on duplicates — the heap is a multiset and pops equal values independently.
- **Rung 4 — three scalars, `O(n)` / `O(1)`.** The answer only ever depends on the top three values seen so far (§2.1):

```
std::ios_base::sync_with_stdio(false);
std::cin.tie(nullptr);

int n; std::cin >> n;
int m1 = 0, m2 = 0, m3 = 0;           // A_i >= 1, so 0 is a safe sentinel
for (int i = 0; i < n; ++i) {
    int x; std::cin >> x;
    if      (x > m1) { m3 = m2; m2 = m1; m1 = x; }
    else if (x > m2) { m3 = m2; m2 = x;          }
    else if (x > m3) { m3 = x;                   }
    if (i >= 2) std::cout << m3 << '\n';
}
```

Review of `c.cpp`. The algorithm is right; three things are worth changing. (1) `std::endl` on up to `5×10^5` lines with no `sync_with_stdio(false)` — §2.9, the only item here with a plausible path to TLE. (2) A fresh `std::vector<int> rem` is constructed inside the loop, so the program performs ~`5×10^5` heap allocations to hold two integers; two locals do the job. (3) `std::vector` is used but only `<iostream>` and `<queue>` are included — it compiles on libstdc++ because `<queue>` pulls in `<vector>` transitively, but that is an implementation detail, not a guarantee. Include what you use.

D — Automat

- **Rung 1 — subsets.** `2^{N+M}` with `N, M ≤ 2×10^5`. Only useful as a correctness oracle on tiny inputs (which is how the greedy below was validated).
- **Rung 2 — model the money correctly.** This is the whole problem, and it is a modelling step, not an algorithmic one.
  - A drink costs `B_j` but the machine takes **only** K-dollar bills, so it consumes `ceil(B_j / K)` of them and returns `ceil(B_j/K)·K − B_j` **in 1-dollar bills**.
  - A dessert accepts either kind, and also returns change in 1-dollar bills.
  - Because every machine gives change, and change is always in the universally accepted denomination, **money is fungible**: any purchase set whose total price is at most `X + K·Y` can be paid for, provided the bills exist when needed — and buying the drinks first guarantees that, since drinks only ever *add* 1-dollar bills to the pile.

  So the problem is: maximise `|chosen|` subject to **(i)** `Σ_{chosen drinks} ceil(B_j/K) ≤ Y` and **(ii)** `Σ_{chosen} price ≤ X + K·Y`. Nothing else.
- **Rung 3 — sort, then sweep with a binary search.** For a fixed number of drinks `r`, the `r` cheapest drinks minimise both (i) and (ii) at once, because `ceil(·/K)` is monotone in the price (§2.2). Same for desserts and (ii). So sort both ascending, prefix-sum `A`, `B`, and `ceil(B/K)`, then for each `r` with `preK[r] ≤ Y`, binary-search the largest `l` with `preA[l] ≤ X + K·Y − preB[r]` and take `max(l + r)`. `O((N+M) log(N+M))`.
- **Rung 4 — two pointers.** `l` is non-increasing in `r`, so the binary search becomes a pointer walking left. The sort still dominates, so this is a constant- factor win, not a complexity one.

Overflow is the live hazard here, not the algorithm. `K·Y` reaches `10^9 × 10^9 = 10^18`; adding `X ≤ 10^15` stays inside `int64` (max ≈ `9.2×10^18`) but leaves no room for an incautious intermediate. `preK` reaches `2×10^5 × 5×10^8 = 10^14`, so the K-bill prefix sums need `long long` as well — that array is easy to leave as `int` because its *elements* look small.

Verification: implemented and checked against samples 1 and 2, then against an exhaustive-subset brute force on 300 random inputs (`N, M ≤ 4`, `K ≤ 7`) — no mismatches.

E — Min-Max Swap

- **Rung 1 — simulate literally.** Scan `P[L..R]` for the min and max, swap: `O(N)` per operation, `O(NM) = 4×10^{10}` worst case. Correct, hopeless.
- **Rung 2 — two segment trees.** One over `(value, index)` merged by larger value, one merged by smaller value (§2.4). Each operation is: query both over `[L, R]`, swap `P[lo]` and `P[hi]`, then point-update **both** trees at **both** positions. `O(N + M log N)`.

There is no cleverer structure to find here, and that is the lesson: E is a data-structure exercise, not an insight problem. The only thing to get right is remembering that a swap dirties two positions in

two trees — four updates, not two. Because `P` is a permutation the extrema are unique, so no tie-break policy is needed; if values could repeat, the operation would be ambiguous and the statement would have had to pick one.

F — Chebyshev Cafe

- **Rung 1 — direct.** `f(i,j)` as written is a sum over all `N²` cells, for each of `N²` cells: `O(N⁴) = 5×10^{12}` at `N = 1500`.
- **Rung 2 — the layered-square identity.** `max(|dr|,|dc|) ≤ t` describes the square of Chebyshev radius `t`, so `max(|dr|,|dc|) = Σ_{t=0}^{N−2} (1 − [max ≤ t])`, and `f(i,j) = (N−1)·T − Σ_{t=0}^{N−2} S(i,j,t)` where `T` is the total population and `S(i,j,t)` is the population inside the radius-`t` square, `O(1)` from a 2D prefix sum. That is `O(N)` per cell, `O(N³) ≈ 3.4×10^9` overall — still too slow at a 3-second limit, but it is the first rung that is recognisably the right *kind* of idea, and it is the standard move for `max`-shaped kernels.
- **Rung 3 — rotate, and the 2D kernel separates.** Using `max(|dr|,|dc|) = (|du| + |dv|)/2` with `u = r+c`, `v = r−c` (§2.5):

  1. Build `W1[u] = Σ_{r+c=u} C[r][c]` and `W2[v] = Σ_{r−c=v} C[r][c]`. Both have `O(N)` buckets.
  2. Prefix-sum each of `W1`, `W2` and `u·W1`, `v·W2`, giving `A1(S)` and `A2(S)`, the weighted absolute-difference sums, in `O(1)` per query (§2.6).
  3. `f(i,j) = (A1(i+j) + A2(i−j)) / 2`.

  `O(N²)` total, dominated by building `C` itself. **Add the two halves before halving** — see the parity note in §2.5.

The editorial reaches `O(N²)` by a different route: it writes the kernel's 2D second difference, which turns out to be supported only on a diagonal and an anti-diagonal, then recovers `f` with cumulative sums taken in two diagonal directions followed by an ordinary 2D prefix sum. Same complexity; the rotation is the one you can re-derive at the board.

Sizing. `C[r][c] < M ≤ 2×10^6`, so `T ≤ 2.25×10^6 × 2×10^6 = 4.5×10^{12}` and `f ≤ T·(N−1) ≈ 6.75×10^{15}` — comfortably `int64`, but well past `int`. The `+ (i−1)N + (j−1)` and the final XOR are just an output-compression wrapper so the judge can check `N²` numbers with one; they do not touch the algorithm. The generous 2048 MiB limit is the tell that an `O(N²)` table of 64-bit values (~18 MB, or more if you keep several) is the intended shape.

Verification: the rotation identity was checked against all three official samples (`467`, `1021`, `764668547`) and, on sample 1, against an `O(N⁴)` direct computation.

G — Increasing Popcount

- **Rung 1 — construct greedily per query.** Walk `L..R` assigning the smallest legal value to each index. Correct, `O(R−L)` per query — fine for the tiny samples, impossible at `R ≤ 10^18`.
- **Rung 2 — reformulate with Dilworth (§2.7).** Value classes are chains in the order `i < j ⇔ i<j ∧ popcount(i)<popcount(j)`, so the answer is the minimum chain cover, which equals the maximum antichain: **the longest non-increasing-popcount subsequence of `[L, R]`**.
- **Rung 2½ — the wrong shortcut.** "Take the biggest popcount class" matches samples 1–3 and is wrong; `[7,8]` breaks it (see §2.7). Worth internalising, because a formula that passes the visible samples is the expensive kind of wrong.
- **Rung 3 — linear DP.** Over `x = L..R` in order, keep `dp[k]` = longest non-increasing subsequence ending at popcount `k`; the transition is `dp[pc] = max(dp[pc], 1 + max_{j ≥ pc} dp[j])`, kept `O(1)` amortised with a suffix-max array. `O((R−L)·α)`. This is what confirms the reformulation empirically — it reproduces sample 4 (`L=100, R=10^7 → 1636985`) — and it is still exponential in the input size.
- **Rung 4 — aligned blocks (§2.8).** Split `[L, R+1)` into `O(log R)` aligned blocks. Each block has a fixed prefix popcount `p` and `c` free low bits, and contributes at most one popcount class, of size `C(c, k)` — that "at most one class" is the LYM lemma, and it is the non-obvious step. DP left to right over blocks with the chosen popcounts non-increasing: `O(log² R)` per query, ~3600 operations, comfortable at `T = 10^4`.

Verification: rungs 2 and 3 were checked against samples 1–3 and, separately, against an independent greedy minimum-chain-cover computation on seven ranges including `[7,8]`, `[1,40]`, `[17,63]` and `[100,300]`; rung 3 reproduces sample 4 exactly. Rung 4 is the editorial's and is **not** independently implemented here — it is reported, not verified.

---

4. Key ideas that would have helped

- **D is a modelling problem in greedy clothing, and it was the reachable one.** Everything hard about it is the sentence "both vending machines give change using only 1-dollar bills". Once you see that change makes money fungible and leaves `Σ ceil(B_j/K) ≤ Y` as the *only* second constraint, what is left is sort
  - prefix sums + a sweep — material already used in ABC 475 C. This is the one unattempted problem in the set that was a write-out-the-invariant away, not a new-technique away.
- **E costs nothing but a segment tree you have not written yet.** `N, M ≤ 2×10^5` with "find the extremum in a range, then change it" has exactly one answer, and the only design decision is storing `(value, index)` instead of `value`. If ABC 475 E's write-up put a Fenwick tree in reach, this is the companion piece: build the `(value, index)` argmax tree once and keep it.
- **`max(|Δx|, |Δy|)` inside a summation means rotate.** F is unreachable without that reflex and near-mechanical with it. The tell is that the `max` is what blocks separability — any identity that removes the `max` is worth trying, and the rotation removes it exactly.
- **Check a candidate formula against a case the samples do not cover.** G's "largest popcount class" passes samples 1, 2 and 3. Two minutes constructing `[7,8]` by hand — a high-popcount number followed by a low-popcount one — kills it. The samples are not a test suite.
- **`T ≤ 10^4` with `R ≤ 10^18` is a per-query budget, stated in the input format.** It reads as "polylog per query", i.e. roughly `60²`. Reading `T` and the bound together, before thinking about the problem at all, tells you what shape of solution to look for.

5. Reading the constraints

#	limits	what that was signalling
A	`1 ≤ |S| ≤ 10`	no algorithm — the statement *is* the solution
B	`1 ≤ N ≤ 100`	anything polynomial passes; write the clearest thing
C	`3 ≤ N ≤ 5×10^5`, `A_i ≤ 10^9`	`O(n log n)` at worst — and `N` output lines, so I/O is part of the budget. `A_i ≤ 10^9` fits `int`; the answer is an element, not a sum, so no `long long` needed
D	`N, M ≤ 2×10^5`, `K ≤ 10^9`, `X ≤ 10^{15}`, `Y ≤ 10^9`	`O(n log n)` — sort + sweep. `X + K·Y ≈ 10^{18}` is the loudest signal in the set: it is an explicit instruction to use 64-bit arithmetic and to check it stays under `9.2×10^{18}`
E	`N, M ≤ 2×10^5`, `P` a permutation	`O((N+M) log N)` — per-operation range extremum with updates is a segment tree. "Permutation" removes tie-breaking
F	`N ≤ 1500`, `M ≤ 2×10^6`, 3 s, **2048 MiB**	`N² = 2.25×10^6`, `N³ = 3.4×10^9`, `N⁴ = 5×10^{12}`. So `O(N²)` or `O(N² log N)`; the doubled memory limit says "build the `O(N²)` table"
G	`T ≤ 10^4`, `L ≤ R ≤ 10^{18}`	`10^{18} ≈ 2^{60}`: polylog per query, `O(log² R)` ≈ 3600 ops. Bit-decomposition plus a DP over bit positions

6. Post-mortem

Where the time went: not recorded.

What to drill:

1. **A `(value, index)` segment tree**, written from scratch, with point update and range argmax/argmin (§2.4). It unlocks E directly and is the same skeleton as the Fenwick tree from ABC 475 E.
2. **The Chebyshev ↔ Manhattan rotation** (§2.5) — derive it from `max(a,b) = (a+b+|a−b|)/2` once by hand so it is recoverable under time pressure, then do one problem that sums Manhattan distances over a grid.
3. **Dilworth's theorem** (§2.7) as a recognition pattern: "fewest groups such that same-group items are totally ordered" → maximum antichain. G is unapproachable without it and mostly mechanical with it.
4. **D end to end**, as a timed exercise. It is the highest-value unattempted problem in the set and needs no new technique.

Open: D, E, F, G all still unsolved in code — `d.cpp` … `g.cpp` are stubs. §3 gives a full route for each; D and E are the two to write first.